

Natural Language Processing

13. Information Retrieval and Question Answering

Paolo Torroni

Fall 2025

Notice

Credits

The present slides are largely an adaptation of existing material, including slides by [Jurafsky & Martin](#), [Manning](#), [Potts & MacCartney](#), [Black & Mortensen](#), and others, to whom I am very grateful.

Downloading and sharing

A copy of these slides can be downloaded from [virtuale](#) and stored for personal use only. Please do not redistribute.

Table of Contents

- Information Retrieval
- Question Answering

Question Answering and Information Retrieval

QA mostly focuses on questions that can be answered with simple facts expressed in short texts (**factoid questions**)

- *What states border Texas?*
- *Where was Ada Lovelace born?*

Nowadays, QA mostly relies on LLMs prompting. However,

- Responses may be not faithful to the facts (**hallucinations**)
 - Remark: **confidence** is *not* indicative of **probability of correctness**
 - Wrong answers often come with complete certainty
- What about company knowledge bases?
- What about rapidly changing information?

Mainstream solution: generate answer based on relevant context (retrieval)

Information Retrieval

Information Retrieval

- Retrieval of all manner of media based on user information needs
- Such an IR system: **search engine**
- **Ad-hoc retrieval**: uses poses a **query** to a retrieval system and obtains an ordered set of **documents** from some **collection**

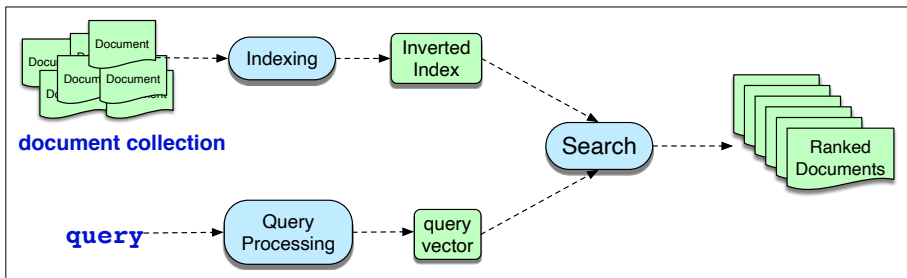


Figure 14.1 The architecture of an ad hoc IR system.

Review: **tf-idf**(t, d) = $tf(t, d) \times idf(t)$

- **tf: term frequency**: frequency count (usually log-transformed)

$$tf_{t,d} = \begin{cases} 1 + \log_{10} Count(t, d) & \text{if } Count(t, d) > 0 \\ 0 & \text{otherwise} \end{cases}$$

- Words that occur many times in a document have a high tf
- **idf: inverse document frequency**

$$idf_t = \log_{10} \frac{N}{df_t}$$

- Words that occur only in a few documents have a high idf
- df_t is the number of documents where t occurs
- Words like *the* or *good* have a low idf

Review: **idf** Illustration with Shakespeare

Word	df	idf
Romeo	1	1.57
salad	2	1.27
Falstaff	4	0.967
forest	12	0.489
battle	21	0.246
wit	34	0.037
fool	36	0.012
good	37	0
sweet	37	0

Review: **tf-idf** Illustration with Shakespeare

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

Figure 6.2 The term-document matrix for four words in four Shakespeare plays. Each cell contains the number of times the (row) word occurs in the (column) document.

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	0.246	0	0.454	0.520
good	0	0	0	0
fool	0.030	0.033	0.0012	0.0019
wit	0.085	0.081	0.048	0.054

Figure 6.9 A portion of the tf-idf weighted term-document matrix for four words in Shakespeare plays, showing a selection of 4 plays, using counts from Fig. 6.2. For example the 0.085 value for *wit* in *As You Like It* is the product of $tf = 1 + \log_{10}(20) = 2.301$ and $idf = .037$. Note that the idf weighting has eliminated the importance of the ubiquitous word *good* and vastly reduced the impact of the almost-ubiquitous word *fool*.

Document Scoring

- Score document **d** by the cosine of its vector **d** with query vector **q**:

$$score(q, d) = \cos(\mathbf{q}, \mathbf{d}) = \frac{\mathbf{q} \cdot \mathbf{d}}{|\mathbf{q}| |\mathbf{d}|}$$

$$score(q, d) = \sum_{t \in \mathbf{q}} \frac{\mathbf{tf-idf}(t, q)}{\sqrt{\sum_{q_i \in q} \mathbf{tf-idf}^2(q_i, q)}} \cdot \frac{\mathbf{tf-idf}(t, d)}{\sqrt{\sum_{d_i \in d} \mathbf{tf-idf}^2(d_i, d)}}$$

- Considerations:
 - Query words are likely to have a count of 1
 - idf(t)** contributes twice → could drop one **idf(t)** term
 - |q|** is the same for all documents

 Simplified version: $score(q, d) = \sum_{t \in \mathbf{q}} \frac{\mathbf{tf-idf}(t, d)}{|\mathbf{d}|}$

Document Scoring Example

Query: sweet love

Doc 1: Sweet sweet nurse! Love?

Doc 2: Sweet sorrow

Doc 3: How sweet is love?

Doc 4: Nurse!

word	Document 1					Document 2				
	cnt	tf	tf-idf	n'lized	$\times q$	cnt	tf	tf-idf	n'lized	$\times q$
sweet	2	1.301	0.163	0.357	0.137	1	1.000	0.125	0.203	0.0779
nurse	1	1.000	0.301	0.661	0	0	0	0	0	0
love	1	1.000	0.301	0.661	0.610	0	0	0	0	0
how	0	0	0	0	0	0	0	0	0	0
sorrow	0	0	0	0	0	1	1.000	0.602	0.979	0
is	0	0	0	0	0	0	0	0	0	0
$ d_1 = \sqrt{.163^2 + .301^2 + .301^2} = .456$						$ d_2 = \sqrt{.125^2 + .602^2} = .615$				
Cosine: $\sum$ of column: 0.747						Cosine: $\sum$ of column: 0.0779				

Figure 14.2 Computation of tf-idf cosine score between the query and nano-documents 1 (0.747) and 2 (0.0779), using Eq. 14.4, Eq. 14.5, Eq. 14.6 and Eq. 14.9.

Document Scoring Example

Query: sweet love

Doc 1: Sweet sweet nurse! Love?

Doc 2: Sweet sorrow

Doc 3: How sweet is love?

Doc 4: Nurse!

$$\text{score}(q, d) = \sum_{t \in q} \frac{\text{tf-idf}(t, d)}{|d|}$$

Doc	$ d $	tf-idf(sweet)	tf-idf(love)	score (simplified)
Doc 1	.456	.163	.301	1.017
Doc 2	.615	.125	0	.203
Doc 3	.912	.125	.301	.467
Doc 4	.301	0	0	0

Okapi BM25

$$BM25(t, d) = \sum_{t \in q} \text{idf}(t) \frac{\text{tf}(t, d) \cdot (k + 1)}{k \cdot \left(1 - b + b \left(\frac{|d|}{|d_{avg}|}\right)\right) + \text{tf}(t, d)}$$

- Another popular **tf-idf variant**
- Adds two parameters
 - k to adjust balance between **tf** and **idf**
 - b to control importance of document length normalization
 - d_{avg} is the average document length
 - Typical values: $k = [1.2, 2]$; $b = 0.75$

The Probabilistic Relevance Framework: BM25 and Beyond, S Robertson and H Zaragoza, 2009

Inverted Index

- Basic search problem in IR is to find all documents $d \in C$ that contain a term $q \in Q$
- **Inverted index** to make search efficient
 - **dictionary** for efficient access
 - **postings**: lists of doc IDs with info such as tf or even term positions

how	{1}	→	3	[1]
is	{1}	→	3	[1]
love	{2}	→	1 [1] → 3 [1]	
nurse	{2}	→	1 [1] → 4 [1]	
sorry	{1}	→	2 [1]	
sweet	{3}	→	1 [2] → 2 [1] → 3 [1]	

Metrics for Information Retrieval

- Use **precision** and **recall**
- Assume each retrieved document is either **relevant** or **not relevant**
 - T : ranked documents returned by system
 - $R \subseteq T$: relevant
 - $N \subseteq T$: those irrelevant
 - $U \supseteq R$: all relevant documents

$$Precision = \frac{|R|}{|T|} \quad Recall = \frac{|R|}{|U|}$$

- How about *ranking* performance?
- Rank-specific precision/recall

Precision Recall Curve

Rank	Judgment	Precision _{Rank}	Recall _{Rank}
1	R	1.0	.11
2	N	.50	.11
3	R	.66	.22
4	N	.50	.22
5	R	.60	.33
6	R	.66	.44
7	N	.57	.44
8	R	.63	.55
9	N	.55	.55
10	N	.50	.55
11	R	.55	.66
12	N	.50	.66
13	N	.46	.66
14	N	.43	.66
15	R	.47	.77
16	N	.44	.77
17	N	.44	.77
18	R	.44	.88
19	N	.42	.88
20	N	.40	.88
21	N	.38	.88
22	N	.36	.88
23	N	.35	.88
24	N	.33	.88
25	R	.36	1.0

Figure 14.4 Rank-specific precision and recall values calculated as we proceed down through a set of ranked documents (assuming the collection has 9 relevant documents).

Precision Recall Curve

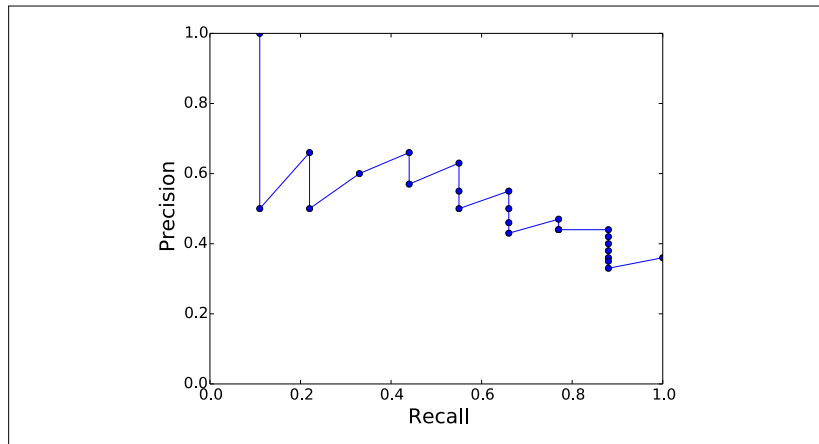


Figure 14.5 The precision recall curve for the data in table 14.4.

Precision Recall Curve

- To evaluate system on multiple queries: **interpolated precision** values for fixed recall levels
- Choose max precision value at any recall level at or above the one we're calculating

$$IntPrecision(r) = \max_{i \geq r} Precision(i)$$

Precision Recall Curve

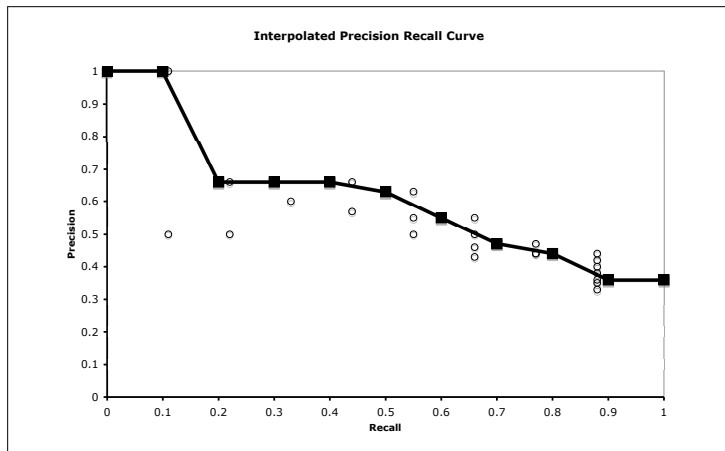


Figure 14.7 An 11 point interpolated precision-recall curve. Precision at each of the 11 standard recall levels is interpolated for each query from the maximum at any higher level of recall. The original measured precision recall points are also shown.

Precision Recall Curve

- Interpolation scheme helps smooth over the irregular precision values in the original data
- Enables comparing systems or approaches by comparing their curves
 - system higher in precision toward the left may favor precision over recall
 - systems more geared towards recall will be higher at higher levels of recall (to the right)

Mean Average Precision

- Alternatively, descend through the ranked list and note the precision only at those points where a relevant item has been encountered
- Evaluate for an ensemble of queries Q

$$\text{average precision } AP = \frac{1}{|R_r|} \sum_{d \in R_r} \text{Precision}_r(d)$$

$$\text{mean average precision } MAP = \frac{1}{|Q|} \sum_{q \in Q} AP(q)$$

IR with Dense Vectors

- Problem with sparse embeddings: **vocabulary mismatch**
 - You search for *tragic love story*, but Shakespeare writes about *star-crossed lovers*
 - It's a conceptual problem: a user posing a query needs to figure the wording used by the writers of documents containing answers
- Solution is **dense retrieval**: use dense embeddings to handle synonymy
- Two main approaches to computing similarity score between dense embeddings:
 - 1 Rely on attention of encoder model + linear head to predict similarity
 - 2 Rely on bi-encoder

IR with Dense Vectors

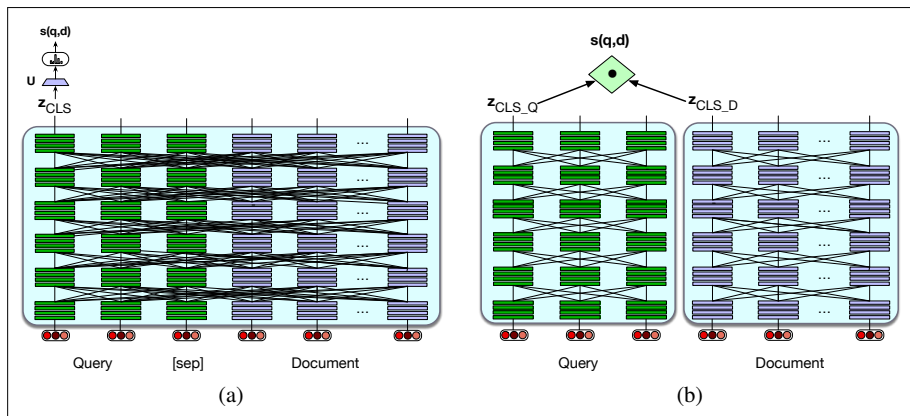


Figure 11.7 Two ways to do dense retrieval, illustrated by using lines between layers to schematically represent self-attention: (a) Use a single encoder to jointly encode query and document and finetune to produce a relevance score with a linear layer over the CLS token. This is too compute-expensive to use except in rescoring (b) Use separate encoders for query and document, and use the dot product between CLS token outputs for the query and document as the score. This is less compute-expensive, but not as accurate.

Dense retrieval with single encoder

- **Cross-encoder**

- Powerful: representation is sensitive to the meaning of both query and document and their interaction

$$\mathbf{z} = \text{BERT}(\mathbf{q}; [\text{SEP}]; \mathbf{d}) [\text{CLS}]$$

$$\text{score}(\mathbf{q}, \mathbf{d}) = \text{softmax}(\mathbf{U}(\mathbf{z}))$$

- Documents are **chunked** to fit, together with the query, in the encoder's window
- BERT and **U** fine-tuned together using a tuning dataset or relevant/non-relevant passages
- Problem: computationally expensive, impractical

Dense retrieval with bi-encoder

- Document encodings computed only once and kept in vector store
- Compute similarity by dot product with query embedding

$$\mathbf{z}_q = \text{BERT}_Q(\mathbf{q})[\text{CLS}]$$

$$\mathbf{z}_d = \text{BERT}_D(\mathbf{d})[\text{CLS}]$$

$$\text{score}(q,d) = \mathbf{z}_q \cdot \mathbf{z}_d$$

- Efficient, but less accurate since can't take advantage of possible meaning interactions
- Alternative: make a shortlist using an efficient method, then **rerank** only the shortlisted documents
- For efficient retrieval, search for nearest neighbour in vector store using approximate K-NN algorithms like **Faiss**

Question Answering

IR-Based Factoid Question Answering

- Task: answering a user's question by finding short text segments from a document collection
- It's a **segmentation** task

Question	Answer
Where is the Louvre Museum located?	in Paris, France
What are the names of Odin's ravens?	Huginn and Muninn
What kind of nuts are used in marzipan?	almonds
What instrument did Max Roach play?	drums
What's the official language of Algeria?	Arabic

Figure 23.9 Some factoid questions and their answers.

- *What steps are involved?*

IR-Based Factoid Question Answering

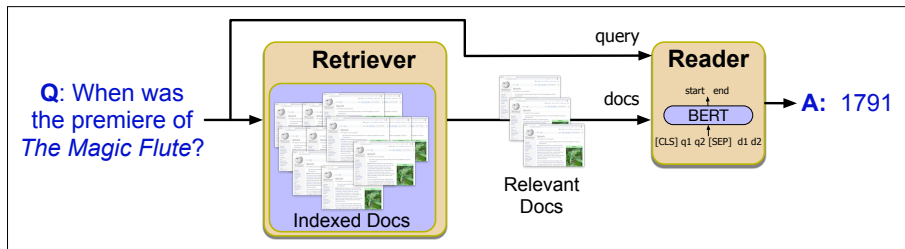


Figure 23.10 IR-based factoid question answering has two stages: **retrieval**, which returns relevant documents from the collection, and **reading**, in which a neural reading comprehension system extracts answer spans.

Dominant paradigm is **retrieve and read**

- 1 Document and passage retrieval by a **search engine**
- 2 Answer extraction by **reading comprehension**

Reading Comprehension Task

- **Reading comprehension task**

- used to evaluate the **reader** component of QA systems
- builds on children's reading comprehension test

- Given:

- A **question** q of l tokens, $q_1, \dots, q_l$
- A **passage** p of m tokens, $p_1, \dots, p_m$

- Compute $\forall p_i$

- $p_{start}(i)$: probability that p_i is the **start of the answer span**
- $p_{end}(i)$: probability that p_i is the **end of the answer span**

Open domain QA refers to the entire retrieve and read task

Answer Span Extraction

- Typically task modeled by **span labeling**

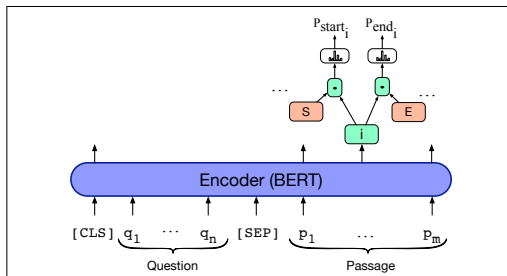


Figure 23.12 An encoder model (using BERT) for span-based question answering from reading-comprehension-based question answering tasks.

- [CLS] for no answer
- Sliding window for longer passages
- Training loss: $L = -\log P_{start_i} - \log P_{end_i}$

Retrieval-Augmented Generation

- The dominant QA paradigm
- Retrieve and generate instead of read (generative, not extractive)
- Generation uses an LLM

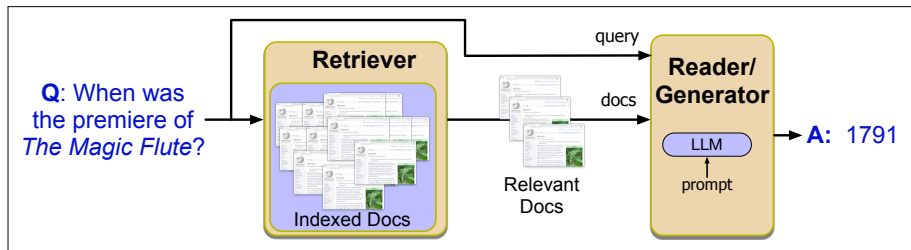


Figure 14.9 Retrieval-based question answering has two stages: **retrieval**, which returns relevant documents from the collection, and **reading**, in which an LLM **generates** answers given the documents as a prompt.

Retrieval-Augmented Generation

- A RAG prompt looks like this:

Schematic of a RAG Prompt

retrieved passage 1

retrieved passage 2

...

retrieved passage n

Based on these texts, answer this question: Q: Who wrote the book "The Origin of Species"? A:

Retrieval-Augmented Generation

- Many **tutorials**, **blogs** and **cookbooks** online (hot topic!)
- Popular tools include LangChain and Llamaindex
- RAG pipelines can include additional stages or use **agents**
- Benchmarking is crucial (and **challenging**), trend is to use **LLMs**

Question Answering Datasets

Lots of IR and QA datasets, closed and open book, mostly generative

- **SQuAD**: 100K questions *crowdsourced* from Wikipedia pages
 - **SQuAD 2.0** also contains 50K *unanswerable* questions
- **MS MARCO** (Microsoft Machine Reading Comprehension), 1M real anonymized Bing query logs + human-generated answers + passages
- **Natural Questions**: real anonymized Google queries; has paragraph-length long answer plus short span answer or no answer
- **MMLU** 14K exam questions (knowledge and reasoning)
- **HotpotQA** created by asking crowdworkers to come up with questions requiring reasoning over multiple documents (provided)
- **TriviaQA** written by trivia enthusiasts, without having a passage in mind. 650K question-answer-evidence triples
- **TyDi QA**: 204K multi-lingual QA pairs
- **CoQA**: conversational QA (free-form answers), **NewsQA**, **WikiQA**, ...

Rogers et al, **QA Dataset Explosion: A Taxonomy of NLP Resources for Question Answering and Reading Comprehension**, 2022

Stanford Question Answering Dataste (SQuAD)

Beyoncé Giselle Knowles-Carter (born September 4, 1981) is an American singer, songwriter, record producer and actress. Born and raised in [Houston, Texas](#), she performed in various [singing and dancing](#) competitions as a child, and rose to fame in the late 1990s as lead singer of R&B girl-group Destiny's Child. Managed by her father, Mathew Knowles, the group became one of the world's best-selling girl groups of all time. Their hiatus saw the release of Beyoncé's debut album, *Dangerously in Love* ([2003](#)), which established her as a solo artist worldwide, earned five Grammy Awards and featured the Billboard Hot 100 number-one singles "Crazy in Love" and "Baby Boy".

Q: "In what city and state did Beyoncé grow up?"

A: "[Houston, Texas](#)"

Q: "What areas did Beyoncé compete in when she was growing up?"

A: "[singing and dancing](#)"

Q: "When did Beyoncé release *Dangerously in Love*?"

A: "[2003](#)"

Figure 25.6 A (Wikipedia) passage from the SQuAD 2.0 dataset ([Rajpurkar et al., 2018](#)) with 3 sample questions and the labeled answer spans.

Massive Multitask Language Understanding (MMLU)

MMLU examples

College Computer Science

Any set of Boolean operators that is sufficient to represent all Boolean expressions is said to be complete. Which of the following is NOT complete?

- (A) AND, NOT
- (B) NOT, OR
- (C) **AND, OR**
- (D) NAND

College Physics

The primary source of the Sun's energy is a series of thermonuclear reactions in which the energy produced is c^2 times the mass difference between

- (A) two hydrogen atoms and one helium atom
- (B) **four hydrogen atoms and one helium atom**
- (C) six hydrogen atoms and two helium atoms
- (D) three helium atoms and one carbon atom

International Law

Which of the following is a treaty-based human rights mechanism?

- (A) The UN Human Rights Committee
- (B) The UN Human Rights Council
- (C) The UN Universal Periodic Review
- (D) The UN special mandates

Prehistory

Unlike most other early civilizations, Minoan culture shows little evidence of

- (A) trade.
- (B) warfare.
- (C) the development of a common religion.
- (D) **conspicuous consumption by elites.**

Figure 11.10 Example problems from MMLU

Metrics for Question Answering

- For **reading comprehension** systems, two common metrics: exact match and F1
- For **multiple-choice** questions like MMLU: **exact match** is % of predicted answers that match gold answer exactly
- For **free text** answers, **F₁ score** measures partial string overlap by treating prediction and gold as bag of tokens, return macro

Metrics for Question Answering

- If system's output is **short ranked list of answers** or passages for each question in the test set: **mean reciprocal rank (MRR)**

$$MRR = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{rank_i}$$

$\frac{1}{rank_i}$ is reciprocal of **rank of first correct answer** of question q_i

- Example: five answers, but the first 3 wrong: RR for that question is $1/4$
- If no correct answer: score 0
- Average over test set Q (or, sometimes, the subset Q of test set questions that have non-zero scores)

Large Language Models for IR

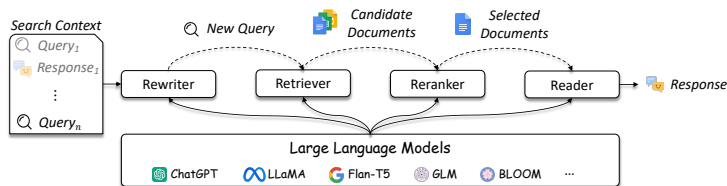


Fig. 1. Overview of existing studies that apply LLMs into information retrieval. LLMs can be used in query rewriter, retriever, reranker, and reader.

- **Retriever** optimized for recall
- **Reranker** for fine-grained reordering of retrieved documents
 - enables tailored ranking strategies
- **Reader** for user intent comprehension and response generation
 - organization of answer texts
 - integration of references

Large Language Models for Information Retrieval: A Survey, Y Zhu et al 2023

Questions?